When Must an Agent Remember?

Bounded Action-Time Memory and Consequential Tool Calls in a Synthetic RFQ Workflow

Mert Turna
Derya

Abstract

Under bounded action-time memory access, benchmark-provided compact current state improved consequential tool-use accuracy relative to model-driven search over append-only episodic history. Under the earlier rereadable-history regimes, this advantage was not apparent. M1 showed that agents can fail to consult memory: GPT-4.1's misses were episodes in which it never called `search_memory`. M2 removed memory-seeking as a confound with a mandatory pre-action lookup gate; M2/M3 showed little observed advantage for answer-key labels or current state once complete relevant history was available. The M4 pilot met its preregistered decision rule. In the final run, oracle versus interactive accuracy was Claude Sonnet 5 60/60 versus 53/60 and GPT-4.1 56/60 versus 35/60. The final set comprises 20 independent base scenarios, each represented at 3 history lengths (20, 50, and 100), for 60 evaluation instances. The paired scenario-cluster bootstrap, added after preregistration to respect scenario non-independence, estimated oracle-minus-interactive accuracy differences of +0.117 [0.017, 0.250] for Claude Sonnet 5 and +0.350 [0.183, 0.500] for GPT-4.1. For the preregistered oracle-versus-interactive comparison, discordant pairs (oracle-only vs interactive-only) were 7 vs 0 for Claude and 24 vs 3 for GPT-4.1; Holm-adjusted exact McNemar p-values were 0.016 and < 0.001. The preregistered instance-level bootstrap and exact McNemar tests with Holm correction are secondary inference. This is descriptive evidence, not a causal mechanism test. The benchmark is synthetic, authored, character-bounded, and limited to two models with one run per episode. It does not evaluate an automatic state updater or establish production performance.

1 Introduction

An operational agent may receive a long sequence of messages and later be asked to send a quote, place an order, or confirm a delivery date. The relevant facts can change during that sequence. A supplier can withdraw an estimate without replacing it; a customer can revise a delivery request for one job but not another; a newer message can concern a different material or condition. A correct action therefore depends on both retrieval and interpretation: the agent must find the applicable evidence, distinguish it from superficially similar records, and act only when the resulting state supports a unique decision.

This paper asks a narrow question: *when does explicitly maintained current-state information improve an agent's consequential tool calls relative to access to historical records?* We do not test a general-purpose memory architecture or claim that any one representation is universally superior. The experiments use a controlled, synthetic RFQ workflow with authored event streams

and deterministic policy evaluators. The experimental contrast is between information conditions at action time, not between independently trained memory systems.

The studies distinguish two regimes. M1 showed a consultation failure: GPT-4.1's misses were episodes in which it never called memory search. M2 introduced a mandatory pre-action lookup gate to remove memory-seeking as a confound. In M2/M3, answer-key labels or current state gave little observed advantage once complete relevant history was available. M4 and the final validation then tested bounded action-time access. The final current-state condition outperformed interactive retrieval, but this is a result about the benchmark's information bottleneck and oracle-like upper bound, not evidence that an autonomous updater can be built at the same cost or with the same accuracy.

The release makes its evidence inspectable and reproducible offline: frozen manifests, raw and aggregate study artifacts, analysis scripts, a compact episode dataset, uncertainty-aware figures, and a static results explorer. Every paper result macro and every aggregate site chart is generated from `results/final/analysis.json`; the scripts and audit are part of the release.

2 Related work

2.1 Memory and state in language-model agents

Recent agent work has explored summaries, persistent notes, retrieval, and reflection as ways to support behavior over longer interactions. Generative Agents combine a memory stream with retrieval and reflection for simulated social behavior [Park et al., 2023]; Reflexion uses verbal feedback to improve later attempts [Shinn et al., 2023]; MemGPT treats context management as an operating-system-like hierarchy [Packer et al., 2023]. These systems motivate persistent memory as a design space, but they do not isolate the question studied here: whether a faithful, compact representation of changing operational facts alters the final consequential action under a matched task and evidence stream.

2.2 Long context and retrieval

Retrieval-augmented generation combines parametric models with retrieved evidence [Lewis et al., 2020]. Passage retrieval can improve knowledge-intensive generation, while long-context evaluations have documented position-sensitive use of evidence even when it is present in a prompt [Liu et al., 2024a]. The interactive arm uses model-chosen BM25 queries over append-only records under a shared character budget. The fixed-BM25 arm instead uses a frozen query built from the job and request; the model's query argument is ignored, and top-ranked raw records are returned under the same cumulative budget. Both are specific retrieval baselines, not universal models of retrieval. The oracle arm supplies benchmark-derived current-state fields.

Xiong et al. study management of an evolving memory bank of past executions through addition and deletion. They find an experience-following property, with error propagation and misaligned experience replay [Xiong et al., 2026]. STALE studies implicit conflicts in which later observations invalidate earlier memories without explicit negation. It probes state resolution, premise resistance, and implicit policy adaptation, and presents CUPMem, a prototype that strengthens write-time

revision through structured state consolidation and propagation-aware search [Chao et al., 2026]. Our question is different: given a correct, compact current-state representation, when does it improve consequential tool actions relative to searching append-only history under bounded action-time evidence access? We do not build or evaluate a write-time updater.

2.3 Tool use and agent benchmarks

ReAct interleaves reasoning and actions [Yao et al., 2023]; Toolformer studies self-supervised tool-use behavior [Schick et al., 2023]. Benchmarks such as AgentBench, WebArena, and SWE-bench evaluate agents in interactive environments with tool-mediated consequences [Liu et al., 2024b, Zhou et al., 2024, Jimenez et al., 2024]. We complement that line of work with a deliberately narrow decision benchmark: a structured state changes during an authored workflow, and an external deterministic scorer checks whether the final action satisfies the policy. The purpose is controlled comparison of action-time evidence conditions, not broad coverage of real APIs or open-ended task competence.

3 Benchmark design

3.1 Synthetic RFQ workflow

Each episode contains an event stream describing a job, customer requirements, supplier capabilities, and operational updates. Policies cover quote routing, purchase-order supplier selection, and delivery-date feasibility. Events may be scoped to a particular job, customer, material, route step, or condition. The authored cases include changes such as a revised lead time, a revocation without a replacement, an irrelevant newer event, a subset-only update, and combinations of constraints. The final set also contains explicit defer cases in which the hidden state does not support a unique action.

The task prompt presents a work item and exposes a finite set of tools. Agents can retrieve memory where the assigned arm permits it, then submit a consequential tool call or escalate. A deterministic policy implementation defines the unique gold action from the full case state; the scorer recomputes whether the submitted tool and arguments satisfy that policy. This separation keeps the scoring rule independent of model explanations and makes incorrect acts, unnecessary deferrals, and appropriate deferrals distinguishable.

The final benchmark varies the length of the event history and samples quote, purchase-order, and date scenarios. It has 20 independent base scenarios, each instantiated at history lengths 20, 50, and 100, yielding 60 evaluation instances and 360 model-by-arm episodes. The M4 pilot contains 21 scenarios and 126 episodes. These authored scenarios are not a random sample of operational RFQ traffic. The case author was also a study designer; scenario difficulty was not calibrated against real RFQ logs.

3.2 Information conditions

The three final arms isolate different information conditions:

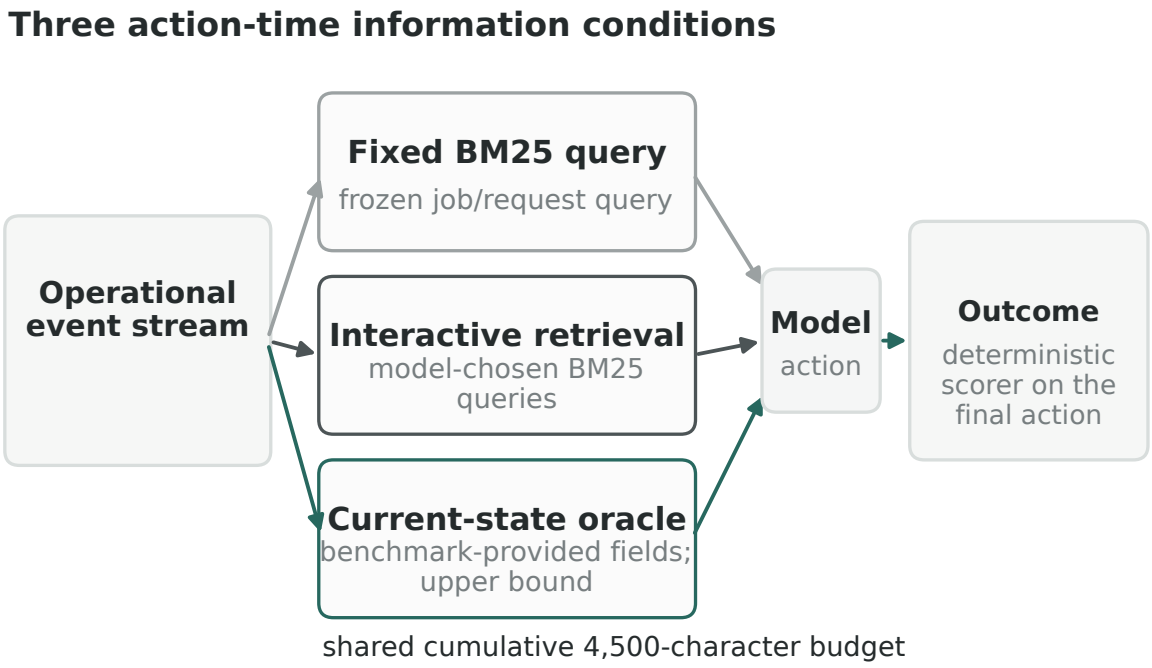


Figure 1: Experimental information conditions. The current-state payload is supplied from benchmark annotations; no automatic updater is under evaluation.

`append_only_fixed` A frozen BM25 query built from the job and request is used; the model's query argument is ignored. Top-ranked raw records are returned under the shared cumulative 4,500-character budget.

`append_only_interactive` Records remain append-only, and the agent can issue model-chosen BM25 queries. Counted returned content consumes the same shared cumulative character budget.

`oracle_current_state_bounded` The benchmark supplies current-state fields derived from the hidden case state in a bounded payload. This is an information upper bound, not an implemented or tested update algorithm.

The oracle condition is intentionally strong. It answers whether access to a compact, correct state representation would change the consequential decision in this task distribution. It does not answer whether a deployed updater can infer that state from noisy records, how much it would cost, or whether it would be reliable in a production workflow.

3.3 Protocol and freeze history

The studies progressed through pilot and validation phases. M1 exposed failures to seek memory; M2 added a mandatory pre-action lookup gate. M2/M3 examined whether answer-key labels or

current state improved decisions when complete relevant history was available. The M4 pilot and final benchmark tested a bounded action-time regime. The final benchmark was preregistered before its scenarios were generated and frozen. The release includes `docs/FINAL-PREREG.md`, the M4 and M4.1 manifests, and the final manifest with checksums.

An aborted M4 run exposed a counter-convergence failure in the bounded-memory accounting path. The correction was a documented post-freeze amendment: the original fixed-point computation was preserved when it converged, and a conservative fallback was used only on nonconvergence. A replay audit compared all persisted memory calls in the completed portion of the aborted run and reported zero mismatches. The follow-up M4.1 frozen run completed under the amended manifest. The amendment changed no scenarios, prompts, tools, retrieval behavior, scorer, or model settings.

For the final benchmark, a six-episode sanity run exercised one long-history instance across the full model-by-arm design before the final freeze. The frozen main run and two sensitivity runs used the same model identities and task definitions; only the memory budget varied for sensitivity. No model API calls are made by the release build, analysis, figures, paper, site, or audit commands. The observed evaluations used one model call sequence per episode; nondeterministic behavior has been observed across earlier reruns even at zero temperature.

4 Results

4.1 Earlier history-access regime

M1 showed that GPT-4.1 can fail to consult memory: its misses were episodes in which it never called `search_memory`. M2's mandatory pre-action lookup gate removed memory-seeking as a confound. In M2/M3, with complete relevant history available, answer-key labels or current state gave little observed advantage. In M3, GPT-4.1 scored 28/35 with append-only history, 26/35 with state labels, and 29/35 with the benchmark's current-state condition. Claude Sonnet 5 scored 35/35, 34/35, and 35/35, respectively. Exact paired McNemar tests comparing each labeled or current-state condition with plain history had p-values of 0.69, 1.0, 1.0, and 1.0; the minimum was 0.69. These observations are specific to the earlier rereadable-history regime; they do not establish equivalence.

4.2 M4 pilot and final validation

The M4 pilot met its preregistered decision rule. In the primary paired contrast, oracle versus interactive accuracy was 21/21 versus 12/21 for Claude Sonnet 5, with oracle-only versus interactive-only discordant pairs 9 versus 0 and exact p=0.0039. For GPT-4.1, the corresponding accuracies were 18/21 versus 6/21, discordances were 13 versus 1, and exact p=0.0018. These results motivated the preregistered final validation.

In the final run, Claude Sonnet 5 scored 47/60 under fixed BM25 retrieval, 53/60 under interactive retrieval, and 60/60 under the current-state condition. GPT-4.1 scored 34/60, 35/60, and 56/60. For oracle versus interactive retrieval, oracle-only versus interactive-only discordant pairs were 7 versus 0 for Claude and 24 versus 3 for GPT-4.1. The Holm-adjusted exact McNemar p-values were 0.016 and < 0.001. Both comparisons favor the oracle condition in this sample.

Table 1: Final overall accuracy by model and information condition.

Model	Fixed	Interactive	Current-state oracle
Claude Sonnet 5	47/60	53/60	60/60
GPT-4.1	34/60	35/60	56/60

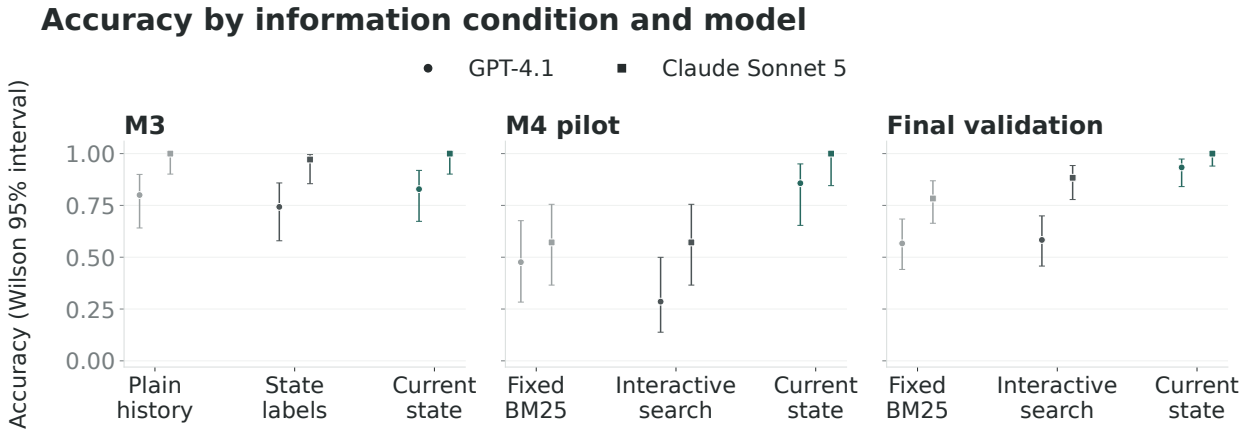


Figure 2: Accuracy by model and information condition for M3, the M4 pilot, and final validation. Error bars are Wilson 95% intervals.

Model-chosen search did not significantly outperform the fixed BM25 query. Interactive versus fixed accuracy was 53/60 versus 47/60 for Claude (interactive-only versus fixed-only discordant pairs: 7 vs 1; p=0.070) and 35/60 versus 34/60 for GPT-4.1 (p=1.0). These null comparisons do not establish equivalence. In the design previews, fixed-BM25 coverage was 245/282 required events in the final set versus 57/78 in the M4 pilot; fixed-BM25 accuracy was also higher in the final set for Claude (47/60 vs 12/21) and GPT-4.1 (34/60 vs 10/21). These coverage and accuracy differences are descriptive, not adjusted comparisons.

4.3 History length and budget

Accuracy by history length and Wilson intervals is shown in Figure 3. For the sensitivity runs, the budget was varied while retaining only the interactive and oracle arms at the longest history length. The main-run budget comparison is taken from the primary run; it is not a separately repeated run. Figure 4 shows the observed accuracy points. The pattern is not uniform across models or conditions: interactive retrieval changes with budget, while the oracle arm is near ceiling in this authored task set. The sample does not identify a universal budget-response curve.

4.4 Failures and paired outcomes

Failure classes are scorer classifications of incorrect final actions, not API failures. Claude's fixed-BM25 misses were 13 (10 `required_record_never_retrieved`, 3 `overcautious_with_evidence`); its interactive misses were 7 (5 required-record misses, 2 overcautious-with-evidence). Claude had 0 oracle misses. GPT-4.1's fixed-BM25 misses were 26 (14 required-record misses, 9 overcautious-

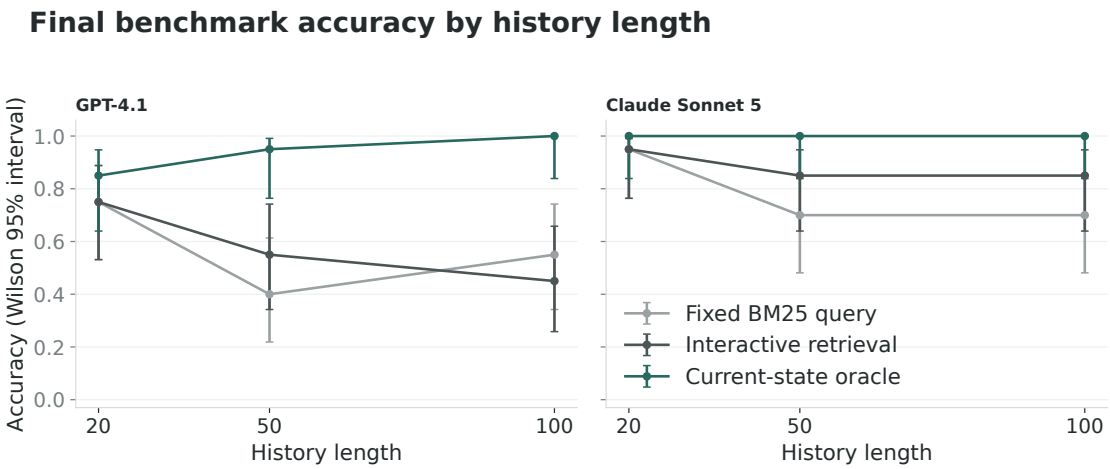


Figure 3: Final accuracy by history length. Each point is a proportion over the episodes in that cell, with Wilson 95% intervals.

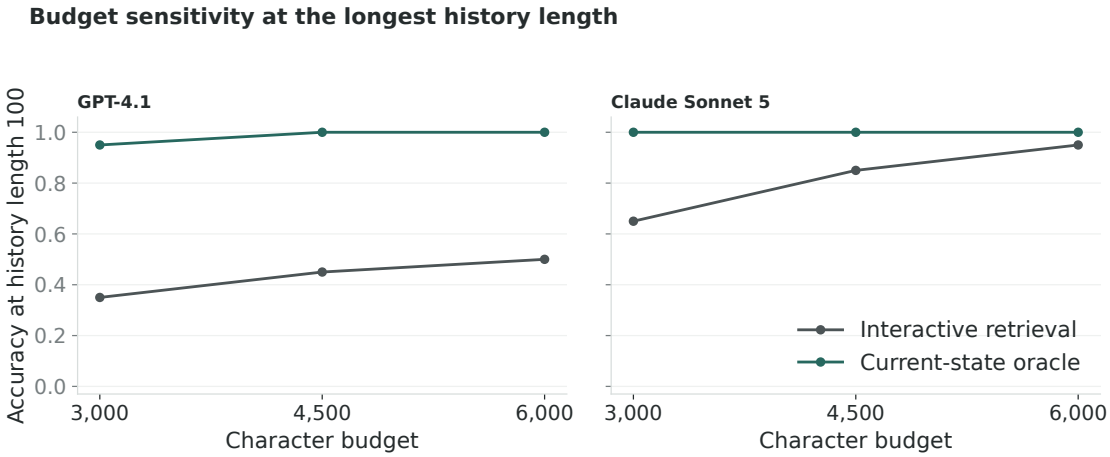


Figure 4: Sensitivity at the longest history length. The main-run point supplies the middle budget.

with-evidence, 3 acted when it should defer). Its interactive misses were 25 (12 required-record misses, 6 overcautious-with-evidence, 5 acted when it should defer, 1 no final action, and 1 stale-value-with-evidence). GPT-4.1 had 4 oracle misses, all 4 `overcautious_with_evidence`; Claude had none. Required records never retrieved were the largest single class of GPT-4.1 interactive misses (12 of 25), not a majority.

All required records were returned in 42/60 fixed-BM25 episodes for Claude and 42/60 for GPT-4.1; interactive search reached complete required-record coverage in 41/60 and 31/60, respectively. The oracle payload had complete required-record coverage in 60/60 and 60/60. These coverage counts are descriptive. Table 2 reports accuracy conditional on evidence return; the oracle condition is not immune to model error or benchmark artifacts.

The primary uncertainty interval is a paired scenario-cluster bootstrap: the 20 independent base scenarios are sampled with replacement, retaining every length variant within each sampled scenario; 10,000 resamples, seed 0, percentile 95% intervals. This cluster bootstrap was added after the

Table 2: Interactive-arm accuracy by required-evidence availability. These counts are descriptive mechanism evidence, not a causal comparison.

Model	All required evidence returned	At least one required item missing
Claude Sonnet 5	39/41	14/19
GPT-4.1	19/31	16/29

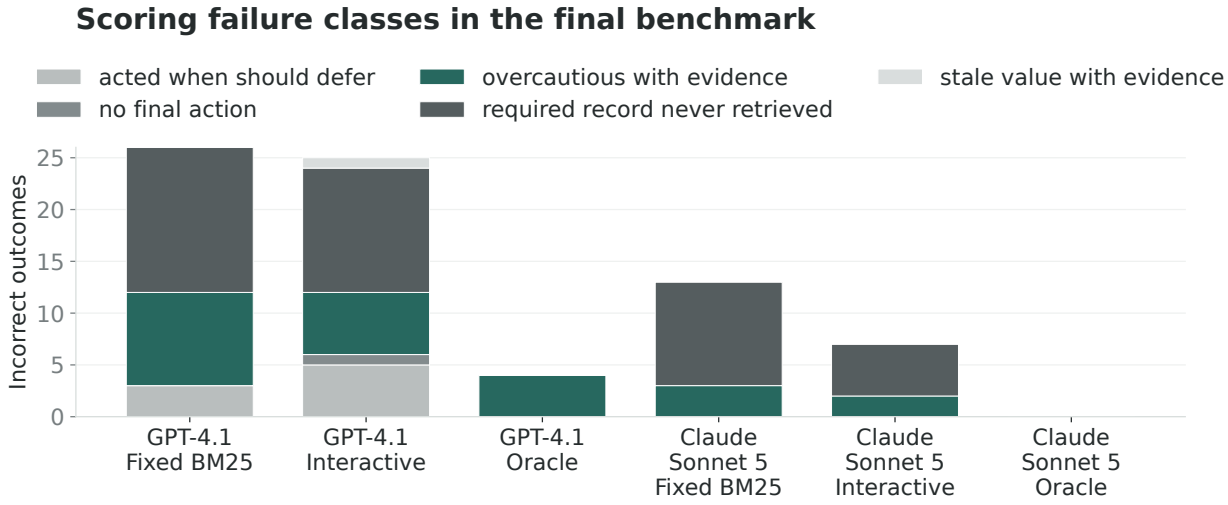


Figure 5: Scored failure classes for incorrect final actions.

preregistered analysis to respect scenario non-independence. Per-model oracle-minus-interactive point differences and 95% cluster-bootstrap intervals are Claude Sonnet 5: +0.117 [0.017, 0.250]; GPT-4.1: +0.350 [0.183, 0.500]. The preregistered instance-level bootstrap and exact McNemar tests with Holm correction are reported as secondary inference. These analyses describe paired outcomes and do not identify a causal mechanism. Secondary comparisons and complete per-cell statistics are included in `results/final/analysis.json`.

5 Discussion

The final result supports a conditional interpretation: under bounded action-time memory access, benchmark-provided compact current state improved consequential tool-use accuracy relative to model-driven search over append-only episodic history. Under the earlier rereadable-history regimes, this advantage was not apparent. M1 showed memory non-consultation, M2 removed that confound with a mandatory lookup gate, and M2/M3 found little observed advantage for answer-key state once complete relevant history was available.

The experiment does not isolate a causal mechanism inside the model. The oracle arm differs in information content and representation, while interactive search requires the model to formulate queries, inspect returned records, and reconcile them. In the interactive arm, Claude was correct on 39/41 episodes when all required evidence was returned, versus 14/19 when at least one required item was missing. GPT-4.1 was correct on 19/31 and 16/29, respectively. These are descriptive

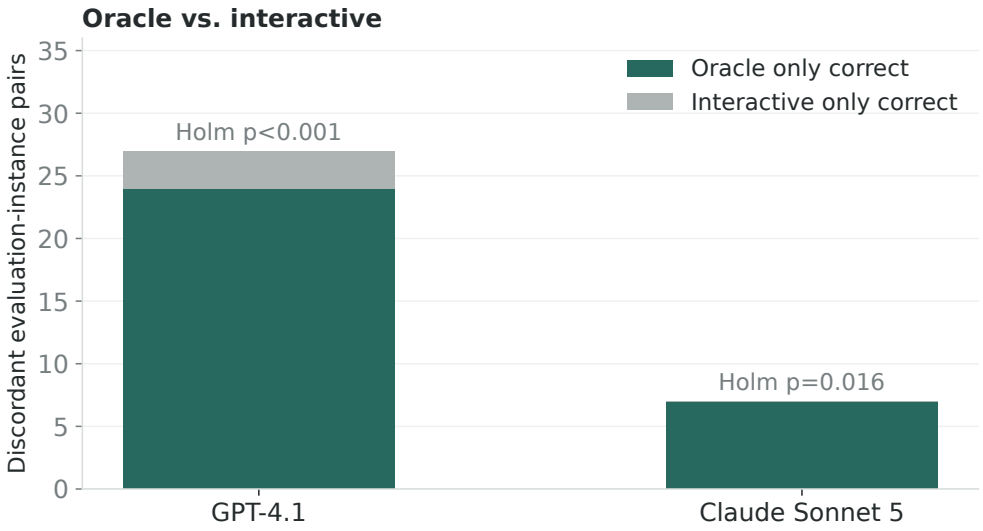


Figure 6: Discordant evaluation-instance pairs for the preregistered oracle-versus-interactive comparison.

mechanism evidence, not causal estimates. In particular, GPT-4.1 was correct in 19/31 episodes despite complete required-evidence return, compared with 56/60 under the oracle. The compact state likely helps beyond evidence availability too, for example by integrating many records; this interpretation is tentative.

6 Limitations

This is a single, synthetic RFQ domain with authored scenarios that were not calibrated to real RFQ logs. It should not be generalized directly to procurement systems or production workflows. The final set has a limited number of defer cases, so estimates for abstention behavior are imprecise and sensitive to case composition. Scenario authorship and scoring conventions may also make some cases easier to solve than naturally occurring work.

Only two model identities were tested, with one run per episode in the final set. Nondeterminism has been observed in the broader project, including at zero temperature; the episode-level sample does not characterize that variance. The fixed character budgets and BM25 retrieval settings are specific design choices, not a model of every context window or search system. Results can change with prompt wording, retrieval configuration, tokenization, or model version. The newest-value mock matches the status-aware reader on the fixed arm; because traps mostly target staleness, this benchmark does not separate variable-aware newest-wins from full state reasoning when relevant records are retrieved.

Most importantly, the current-state condition is a benchmark-provided upper bound. No automatic state-aware updater was built or tested, and the paper makes no claim about its construction cost, feasibility, or reliability. The result does not imply that a state memory guarantees reliable action, nor that a production agent should act without human review.

7 Conclusion

Under bounded action-time memory access, benchmark-provided compact current state improved consequential tool-use accuracy relative to model-driven search over append-only episodic history. Under the earlier rereadable-history regimes, this advantage was not apparent. Research v1 does not build or evaluate an updater; an automatic state updater is future work for Research v2.

A Prompts and tool schemas

The frozen system and task prompts are retained with the study artifacts and identified by the freeze manifests. Across arms, instructions require the model to use available evidence, invoke the assigned memory interface, submit an allowed consequential action, and defer when the policy state does not support a unique action. The user-task text and action policy are held fixed across memory arms. The exact per-episode prompt, provider schema, and request settings are preserved in the run records; source and configuration hashes are in `docs/FREEZE-final.json`.

Tool	Structured arguments and role
`search_memory`	`job_id`; bounded retrieval adds `query`. Returns event records, and in bounded conditions, serialized-length and budget accounting.
`send_quote`	`job_id`, `price_usd`, `lead_time_days`.
`place_purchase_order`	`job_id`, `supplier`.
`confirm_delivery_date`	`job_id`, `delivery_date`.
`escalate`	`job_id`, `reason`; defer or request human review.

The scorer checks structured arguments and policy feasibility; model-generated rationales do not determine the gold label.

B State representation and illustrative case

The oracle payload uses a fixed operational schema that does not depend on the gold action. It always contains eight job fields (`budget_cap_usd`, `requested_date`, `castings_arrival`, `material_spec`, `drawing_revision`, `packaging`, `payment_terms`, `ship_to`), lead and days fields for every step of the job's route, and approved/hold/lead fields for every supplier in the case's supplier registry. Fields are listed in a fixed deterministic order (job fields, then route steps in route order, then suppliers sorted by name), and unset fields appear as "not recorded". The schema therefore contains many non-decisive current fields. Field values come from the benchmark's event-effect annotations, which is what makes the arm an oracle. The payload constructor does not receive the policy, policy parameters, accepted actions, or trap patterns, and no field is selected based on the gold result. An automated test, `test_oracle_schema_and_state_are_case_independent`, in `tests/test_m4_bounded.py`, blanks the policy, policy parameters, accepted actions, and traps and asserts that the serialized oracle payload is byte-identical.

An illustrative quote-route case contains an earlier lead-time estimate, a later correction scoped to the current job, and a newer record for a different job. The correct quote follows the in-scope correction rather than whichever event is newest globally. Full event text, required-event annotations, policies, and accepted actions are in the released case data; the illustrative description here omits operational values.

C Complete accuracy tables

C.1 M3

Model	Plain history		State labels	Current-state condition
GPT-4.1	28/35	26/35	29/35	
Claude Sonnet 5	35/35	34/35	35/35	

C.2 M4 pilot

Model	Fixed BM25 query		Interactive retrieval	Current-state oracle
GPT-4.1	10/21	6/21	18/21	
Claude Sonnet 5	12/21	12/21	21/21	

C.3 Final validation by history length

Model	Arm	N=20	N=50	N=100
Claude Sonnet 5	Fixed	19/20	14/20	14/20
Claude Sonnet 5	Interactive	19/20	17/20	17/20
Claude Sonnet 5	Oracle	20/20	20/20	20/20
GPT-4.1	Fixed	15/20	8/20	11/20
GPT-4.1	Interactive	15/20	11/20	9/20
GPT-4.1	Oracle	17/20	19/20	20/20

Each cell is correct/total; exact denominators and confidence intervals are in the machine-readable analysis JSON and summary CSV.

C.4 Budget sensitivity at the longest history

Model	Arm	3,000 characters	4,500 (main run)	6,000 characters
GPT-4.1	Interactive	7/20	9/20	10/20
GPT-4.1	Oracle	19/20	20/20	20/20
Claude Sonnet 5	Interactive	13/20	17/20	19/20
Claude Sonnet 5	Oracle	20/20	20/20	20/20

D Artifact map

The reproducibility instructions in `REPLICATION.md` give the offline command sequence. The frozen case sets and their manifests are under `data/` and `docs/`; analysis code is under `scripts/`; compact episode and aggregate data are under `results/final/`; figures are under `figures/generated/`; and the no-build explorer is `site/index.html`. The release audit checks generated number consistency, manifests, links, and prohibited claim phrases.

References

Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. STALE: Can LLM agents know when their memories are no longer valid?, 2026. URL https://arxiv.org/abs/2605.06527.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.06770.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024a. doi: 10.1162/tacl_a_00638.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hanyu Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024b. URL https://arxiv.org/abs/2308.03688.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

Joon Sung Park, Joseph O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023. doi: 10.1145/3586183.3606763.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2302.04761.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2303.11366.

Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.27. URL https://aclanthology.org/2026.acl-long.27/.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2307.13854.